Semantics-Directed Conformance Testing using SMT-based Program Synthesis

MEHRAD HAGHSHENAS

In formal verification, it is important to ensure that the implementation faithfully models the specification. A commonly known problem is the gap between the specification and the concrete implementation of a software system, which is a primary source of semantic bugs. This proposal outlines the development of *RuSmart*, a conformance testing framework that bridges this gap by defining the denotational semantics of a language as an executable semantic framework. This executable formal semantics, written in a custom Rust Domain Specific Language, models the language's specified behavior. The denotational semantics will be transpiled into SMT-LIB formulas. We will leverage the Z3 Satisfiability Modulo Theories (SMT) solver as a program synthesizer to systematically synthesize concrete programs that satisfy each path condition of the symbolic interpreter. The generated models will be used as input programs for differential testing to detect divergences between commercial implementations and the reference interpreter of the target language. In essence, our proposal introduces a systematic program synthesis approach for specification testing, using SMT to generate programs targeting specific semantic behaviors, namely so-called *edge cases* and *error conditions*. We will demonstrate this technique on the TOML v1.0.0 specification, with the objective of discovering compliance bugs.

1 Introduction

The correctness of software is paramount for system reliability, particularly in critical systems where unexpected behavior can lead to severe consequences. This correctness becomes more acute in foundational tools like *compilers/interpreters*. A bug in a compiler or interpreter can introduce hidden security vulnerabilities into every piece of software it processes. Therefore, ensuring the reliability of these foundational tools is critical for building trustworthy software. On the engineering of reliable software, *formal verification* offers the utmost correctness assurance to programs, where the correctness of a system is mathematically proven with respect to a certain specification. However, the guaranties provided by formal verification are only as strong as the specification itself; i.e., their reliability depends entirely on how accurately the formal semantics capture the behavior of the system being modeled. In many cases, software components contain informal specifications, written in natural ambiguous language. Even when formal semantics are defined, they are frequently treated as an afterthought. To this purpose, *conformance testing* bridges the gap between a formal specification and its implementation. However, the effectiveness of a testing framework is limited by the quality and completeness of its test suites. We propose a novel testing framework where, we create an executable specification and check for conformance between the implementation and the specification; any divergence discovered through the test suites indicates either a compliance bug in the implementation or a potential flaw in the original specification.

This project will present a framework called *RuSmart* for semantics-directed conformance testing. We will first define the denotational semantics of a target language as a functional interpreter using a custom Domain-Specific Language. In essence, the DSL is inspired by the data types and functionalities available in the API of the Z3 SMT solver. The DSL will enable writing a single Rust program that serves both as an executable interpreter and as a specification that can be translated into SMT-LIB constraints for Z3. Furthermore, the denotational semantics will be manually derived from the documentation of the target language. Crucially, this interpreter would be enriched with fine-grained constructs that explicitly capture the error conditions and edge cases defined in the language specification. We will then transpile the high-level Rust code to an

Author's Contact Information: Mehrad Haghshenas.

intermediate representation and then SMT formulae, where the specification's functions become solvable functions within the backend Z3 SMT solver. We will utilize Z3 as a robust model generator to synthesize input programs that adhere precisely to the formal specification. We further guide the synthesis by asserting targeted path constraints directly to the solver, systematically exercising the error & edge cases and ensuring comprehensive test coverage. The resulting set of synthesized programs will be used to achieve two main goals: 1) They will serve as input programs for the symbolic interpreter itself, providing validation that the transpilation process from Rust to SMT is sound. 2) They will be used as test suites for conformance testing.

2 Work plan

The remainder of the research will be conducted over a two-month period, structured into four phases as follows:

(1) Development (Weeks 1-2)
- **DSL Design.** Systematically review the Z3 C API documentation to identify the full set of functions required for the SMT theories.
- **DSL Implementation.** Develop the standard library crate implementing a complete set of data types and their associated methods. The API must be expressive enough to provide the primitives for the interpreter.

(2) TOML Denotational Semantics (Weeks 3-4)
- **Specification Analysis.** Conduct an analysis of the TOML v1.0.0 specification and its ABNF grammar.
- **Test Identification.** Identify all explicit error conditions and edge cases from the specification text. This will form the basis of our test plan.
- **Semantic Implementation.** Using the DSL, implement the denotational semantics of a TOML parser as a set of pure functions.
- **Semantic Enrichment.** Enrich the symbolic interpreter with fine-grained branches that explicitly model each of the identified error and edge cases.

(3) Transpilation Engine (Weeks 5-6)
- **IR implementation.** Fine-tune the procedural macro-based transpiler. The first stage will be to change the front-end of the transpiler pipeline, transpiling the DSL to the Intermediate Representation (IR); if needed.
- **Complete Transpiler Implementation.** This involves modifying the back-end to convert the IR into SMT-LIB formulas and extract the path conditions for each branch.
- **Synthesis Formulation.** Develop the logic that systematically iterates through the extracted path conditions and formulates the SMT queries required to synthesize programs. In this stage, maybe implementing some heuristics will be needed to fine tune the synthesizing process.

(4) Testing (Weeks 7-8)
- **Self-Validation.** Validate the soundness of the transpilation process by executing the synthesized test cases on the symbolic interpreter.
- **Conformance Testing.** Identify a suite of popular, open-source TOML parsers for differential testing. Run the test suite against the commercial interpreters and log all divergences from the reference interpreter. Finally analyze the results to identify compliance bugs.

Future Work: Following the completion of the TOML case study, future work will focus on demonstrating the *generalizability* of the RuSmart framework. We intend to apply the same methodology to model a declarative language (Rego) and an imperative language (a subset of WebAssembly).

As further work, we also aim to provide a quantitative analysis; i.e., to compare the effectiveness of our approach against grammar-based fuzzing over equivalent time periods.

3 Proposed results

The completion of this project will yield the following artifacts:

(1) **A Framework for Executable Semantics.** The primary artifact will be the `RuSmart` framework, consisting of: (1) a standard library of symbolic types for writing pure functional programs in Rust; (2) a transpiler for converting a subset of rust code into SMT-LIB formulas. Often certification projects assume transpilation soundness without proof. However, we test it by executing synthesized test cases on the symbolic interpreter; and (3) a program synthesizer based on the Z3 solver.

(2) **A Verifiable, Executable Semantics for TOML.** We will produce an executable denotational semantics for the TOML v1.0.0 specification. This implementation, written in our DSL, serves as both a concrete reference interpreter and as the formal model.

(3) **Conformance Testing and Bug Discovery - hopefully :).** The result will be an analysis of the conformance of several open-source TOML parsers. We expect to identify divergences where the commercial implementations deviate from the formal specification.

All in all, this work will provide the programming languages and software testing communities with an open-source toolchain enabling researchers to apply semantics-directed testing to their own implementations.

4 Related work

Our work addresses three key areas: (1 limitations of grammar-based fuzzing, (2) SMT-based program synthesis, 3) frameworks for executable formal semantics.

Differential testing and comparing outputs across multiple implementations to detect divergences has been previously used for compiler testing. Csmith, a randomized test-case generation tool, was used for three years to find compiler bugs; during which more than 325 previously unknown bugs were discovered [Yang et al. 2011]. However, these approaches rely on existing implementations as oracles and cannot target specification-defined edge cases. Our work provides a foundation for differential testing by using executable formal semantics as the oracle. This will allow enumerative testing of specification-defined error conditions and edge cases.

The automated generation of test inputs from a formal grammar has been used for testing compilers and interpreters. An approach widely used in automated test generation is fuzzing, particularly grammar-based fuzzing for structured inputs. While it is effective at generating syntactically valid programs, this approach is limited by its semantic blindness. As demonstrated by [Marmsoler and Brucker 2022], a grammar-based fuzzer understands the syntax of a language but has no knowledge of its semantic meaning. Their work addresses this by creating an *enriched* grammar to guide the fuzzer to generate more meaningful Solidity programs. Their system then performs conformance testing by comparing the execution results from a test oracle, derived from an Isabelle/HOL semantics, against the real Ethereum implementation to find discrepancies. While they do conduct conformance testing, it relies on a guided-random fuzzing process. Nevertheless, inspired by their principle of semantic enrichment, we will directly encode the error conditions and semantic edge cases derived from the TOML v1.0.0[1] specification as fine-grained branches within the symbolic interpreter. This transforms the interpreter into a comprehensive test plan where Z3 can utilize to systematically generate program inputs designed to trigger all explicitly defined **error cases** and **edge cases** derived from the specifications. This *enumerative testing* ensures that every semantic

[1] https://toml.io/en/v1.0.0

rule related to exceptional conditions is thoroughly exercised. The novelty here is in embedding the semantic rules directly into executable specifications, transforming coverage into a systematic SMT-solving problem.

The framework SMT2Test employs SMT formulas as a test program generator, transforming the test cases into imperative programs where the SMT solver's satisfiability result is the test oracle. Our proposed approach, RuSmart, shares the goal of utilizing Z3 as a model generator for generating test inputs. However, SMT2Test, transforms SMT formulas into test programs for verifier testing [Zhang and Su 2024]. Whereas, RuSmart transforms the denotational semantics into SMT constraints to synthesize input programs for conformance testing commercial implementations.

Lastly, the K-framework is a language independent framework, in which the operational semantics of a target language is defined. The system addresses semantic blindness by using an executable specification as the ground truth for a language's behavior, deriving tools like interpreters and compilers automatically from the formal definition [Chen and Roşu 2018]. Figure 1 shows the tools that K-framework automatically generates from the semantics. The idea of our work is primarily inspired by executable formal semantic frameworks, such as the K-framework but pivots the focus: 1) K-Framework focuses on verification of individual programs against operational semantics rather than test generation for conformance testing. 2) Furthermore, we define our specification using denotational semantics (rather than K's operational semantics) which provides a more natural mapping to the functional language of SMT-LIB formulas.

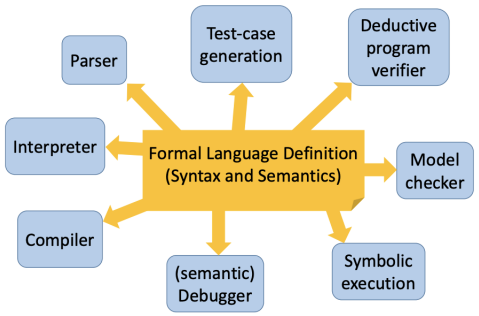

Fig. 1. The picture was taken from [Rosu 2017] and highlights the *correct-by-construction approach*.

References

Xiaohong Chen and Grigore Roşu. 2018. A language-independent program verification framework. In *International Symposium on Leveraging Applications of Formal Methods*. Springer, 92–102.

Diego Marmsoler and Achim D Brucker. 2022. Conformance testing of formal semantics using grammar-based fuzzing. In *International Conference on Tests and Proofs*. Springer, 106–125.

Grigore Rosu. 2017. : A semantic framework for programming languages and formal analysis tools. In *Dependable Software Systems Engineering*. IOS Press, 186–206.

Xuejun Yang, Yang Chen, Eric Eide, and John Regehr. 2011. Finding and understanding bugs in C compilers. In *Proceedings of the 32nd ACM SIGPLAN Conference on Programming Language Design and Implementation* (San Jose, California, USA) *(PLDI '11)*. Association for Computing Machinery, New York, NY, USA, 283–294. doi:10.1145/1993498.1993532

Chengyu Zhang and Zhendong Su. 2024. SMT2Test: From SMT Formulas to Effective Test Cases. *Proc. ACM Program. Lang.* 8, OOPSLA2, Article 279 (Oct. 2024), 24 pages. doi:10.1145/3689719